Relación 3: Arrays

Sitio:	Centros Granada 26	Imprimido por:	Triay Orfila, Marc
Curso:	Desarrollo web en entorno servidor	Día:	martes, 6 de octubre de 2026, 18:22
Libro:	Relación 3: Arrays		

Tabla de contenidos

1. Operaciones básicas

2. Relleno automático

3. Cuadrado y cubo

4. Operaciones aritméticas

5. Complementario

6. Arrays asociativos

7. Borrado

8. Animales

9. Array de arrays

10. Ajedrez

11. Jugando con cartas

11.1. Antes de comenzar a jugar

1. Operaciones básicas

Escribe un script que almacene un array de 8 números enteros. El programa debe realizar de manera secuencial las siguientes operaciones:

a. recorrer el array y mostrar sus elementos por pantalla

b. ordenar de menor a mayor y volver a mostrarlo.

c. Calcular y mostrar en pantalla su longitud total (número de elementos).

d. Buscar un elemento concreto dentro del array de forma estática informando si existe o no.

e. Modificar la búsqueda para que el elemento a localizar se reciba de manera dinámica por la URL empleando parámetros GET.

Para mostrar los elementos del array en cada uno de los apartados se creará una función llamada mostrarArray().

2. Relleno automático

Desarrolla un script que añada de forma automática valores numéricos enteros a un array utilizando un bucle (mientras su longitud sea inferior a 120 elementos).

Una vez rellenado, muestra el contenido completo del array por pantalla.

3. Cuadrado y cubo

Escriba un script PHP que:

- Guarde en un array 20 valores aleatorios entre 0 y 100.
- En un segundo array, llamado cuadrados, deberá almacenar los cuadrados de los valores que hay en el primer array.
- En un tercer array, llamado cubo, se deben almacenar los cubos de los valores que hay en el primer array.
- Por último, se mostrará el contenido de los tres arrays dispuesto en tres columnas paralelas.

4. Operaciones aritméticas

Crea dos arrays independientes con el mismo tamaño (especificado mediante una constante). Rellena el primer array con los números enteros del 1 al N, y el segundo con los números en orden inverso (desde N hasta 1).

Diseña un algoritmo que recorra ambos arrays simultáneamente y genere un tercer array donde cada posición sea el resultado de sumar los elementos equivalentes de los dos arrays anteriores.

5. Complementario

Escribe un script que rellene un array con valores aleatorios (0,1) y lo muestre. A continuación, calcularemos su complementario y también la mostraremos.

Por ejemplo:

```
A: 1 1 1 0 0 1 0 1 0 0
Ā: 0 0 0 1 1 0 1 0 1 1
```

6. Arrays asociativos

a) Crea un array asociativo (o diccionario) con el censo de población de: España, Portugal, Francia, Italia y Grecia.

b) Define un array asociativo cuyas claves sean palabras en castellano y sus valores correspondientes sean las traducciones de dichas palabras en inglés. Genera de forma dinámica, a partir del array, una tabla HTML limpia que muestre el diccionario de sinónimos de manera ordenada en dos columnas.

7. Borrado

Escribe un programa que guarde en un array los nombres de las provincias de Andalucia.

Usa la función unset() para borrar el elemento que desees de ese array. Deberás pasarle a la función como parámetro el índice del elemento a borrar.

Por ejemplo, puedes borrar la de índice 2.

Nota: Si no pones el índice en la función unset() se borrarán todos los elementos del array

8. Animales

Escriba un programa que:

- Muestre un grupo de entre 20 y 30 animales al azar.

Se usará un array para almacenar el código de los animales. El número de elementos del array será el determinado aleatoriamente entre 20 y 30. Tenga en cuenta que los animales son caracteres con rango Unicode: 128000 a 128060. Por ejemplo, podría mostrar el siguiente resultado:



- A continuación, mostrará un animal al azar de los que están incluidos en el grupo anterior y lo eliminará del array. Por ejemplo:



- Por último, mostrará de nuevo el grupo inicial, pero habiendo eliminado del grupo los animales que coincidan con el animal suelto (al menos habrá uno). También mostrará un mensaje con el número total de animales que quedan. En el ejemplo anterior quedarían 21 animales.

Notas:

- uso de las funciones rand(), array_rand(), unset.
- Crea las funciones que consideres oportunas.

9. Array de arrays

Supongamos que guardamos en una array los datos de los profesores de este centro educativo.

Cada elemento de este array a su vez será otro array asociativo que nos permitirá almacenar su número de registro personal, su nombre, su apellido/s, su teléfono, y su fecha de nacimiento.

Se desea:

a) Crear un array con al menos los datos de 3 profesores

b) Crear una función que nos permita mostrar el número de registro personal de cada uno de los profesores

c) Modifica la función anterior y conviértela en una función anónima (usa array_map()).

d) Crea una función anónima que nos permita mostrar los profesores que han nacido a partir de 1990. (Usa strtotime() y array_filter()

NOTA: Ejemplo de uso de funciones anónimas , arrays , array_map() y array_filter().

10. Ajedrez

Se pide que dibujemos un tablero de ajedrez con sus piezas en la posición inicial.

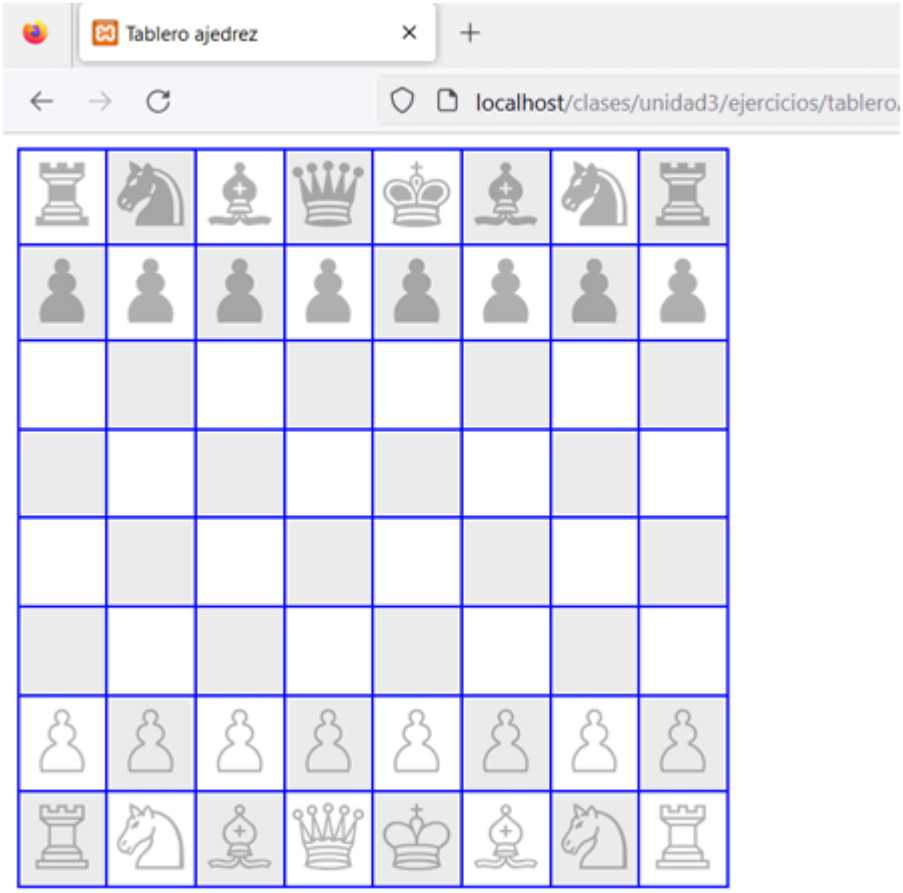
En la parte inferior estarán las piezas blancas, y en la superior, las negras.

Las casillas tendrán fondo blanco o gris, en función de su posición: la casilla inferior derecha desde el punto de vista de las piezas blancas debe ser de color blanco, y a partir de ahí se van alternando casillas blancas y negras.

Las imágenes de las piezas puedes encontrarlas en el archivo adjunto.

Para la resolución de esta actividad se usarán arrays y funciones.

El resultado final debería ser el que se muestra en la siguiente imagen:



Consejos:

- Para facilitar el diseño de la tabla y las piezas, se facilita el siguiente código CSS, que puede usarse en la aplicación:

```
table, td {border:2px solid blue;border-collapse:collapse; }
td { width:50px;height:50px;margin: 0 auto;padding:0px;}
td.blanca {background-color: white;}
td.gris {background-color: #EEEEEE;}
img { opacity: 0.3;width:100%;height:100%;margin:0px;}
```

- Dividir el proceso en diferentes funciones facilita la organización del código y su comprensión.
- Usa funciones para realizar las diferentes tareas, de forma que se minimice la repetición de código.Se aconseja crear un archivo, exclusivamente, para recoger todas las funciones que usaremos en la aplicación.
- A la hora de dibujar el fondo de cada casilla (blanco o gris), hay que tener en cuenta que la casilla inferior derecha es blanca, y, a partir de ella, se van alternando los colores. Si nos fijamos, las casillas blancas son aquellas en las que la suma del número de fila y el número de columna es impar, mientras que, si la suma es par, la casilla es negra/gris.
- Usa arrays para guardar los nombres de las imágenes correspondientes a cada pieza. Se aconseja un array para las blancas y otro para las piezas negras. No hace falta un array para los peones, ya que la figura en todas las casillas de la fila es la misma, aunque puede usarse.

Ejemplo:

```
$piezasBlancas=array("torreb","caballob","alfilb","reinab","reyb","
                alfilb","caballob","torreb");
```

- Intenta que el programa principal quede lo más simplificado posible.

Descargas:

- ***figuras de ajedrez***

11. Jugando con cartas

Las dos actividades propuestas se basan en el uso de arrays para representar las 40 cartas de la baraja española y simulan juegos típicos.

Descarga aquí las imágenes de las cartas.

Y aquí el **reverso** de las cartas por si lo necesitas.

11.1. Antes de comenzar a jugar

- Una baraja española de 40 cartas tiene cuatro palos que son: oros, copas, espadas y bastos.
  - **Crea un array llamado $palos con los palos de la baraja española.**
- De la baraja de 40 cartas, en estos juegos, se utilizan los siguientes números: 1, 2, 3, 4, 5, 6, 7, 10, 11, 12. Los números 8 y 9 no se usarán en la mayoría de los juegos propuestos. Y además algunas cartas representan figuras:
  - 1 representa el As.
  - 10 representa la Sota.
  - 11 representa el Caballo.
  - 12 representa el Rey.
- **Cada carta será un array asociativo** que almacena el número de la carta y el palo.
- La **baraja** completa será también un array que almacena las cartas (**array de arrays**).
  - **Crea una función que genere la baraja completa**. Es decir, devolverá un array con las 40 cartas. Su nombre: `crearBarajaCompleta()` . No olvides que hay que combinar todos los palos con todos los números. No puede haber cartas duplicadas
  - **Crea una función que genere la baraja omitiendo los números 8 y 9 de cada palo**. Su nombre: `crearBaraja()`
- Una vez creada la baraja, es posible que tengamos que barajar las cartas. Para ello:
  - **Crea una función llamada `barajarBaraja(&$baraja)`** que mezcle aleatoriamente la baraja. Se usará la función de PHP `suffle()`. No hay que hacer ningún otro algoritmo dentro de esta función.
- Las cartas "se coloca boca abajo" sobre el tapete y por lo tanto sólo se pueden sacar cartas de la baraja siempre por el extremo final. Nunca se puede sacar una carta de en medio del mazo.

    - **Crea una función llamada** `sacarCarta(&$baraja)` . Cuando una carta ha salido del mazo de la baraja no puede volver a aparecer, se elimina de la baraja.
- Crea una función que muestre la imagen de una carta y su nombre: `mostrarCarta($carta)`
- Simula un tapete poniendo un fondo verde en tu página web

Reiniciar tour para usuario en esta página